SATQUERY AI

An Interactive Vision-Language Assistant for Multimodal Remote Sensing Image Analysis through Text Queries

---

Problem Statement ID: SIH26167
Theme: Space Technology
Organization: Indian Space Research Organisation (ISRO)
Category: Software Track
Document Version: 1.0.0 (Production Grade)
Date: August 2026

1. Executive Summary & Problem Context

Traditional remote sensing frameworks operate as isolated applications designed for static, single-purpose pipelines (e.g., rigid land-cover segmentation or single-category object classification). Geospatial end-users without formal GIS software expertise or deep Python programming capabilities encounter significant entry barriers when trying to unlock on-demand contextual intelligence from raw satellite assets. This structural limitation prevents real-time operations during dynamic, high-stakes scenarios such as active environmental disasters or continuous border reconnaissance.

SatQuery AI completely breaks this architectural bottleneck by establishing a unified, multi-modal, conversational Vision-Language Assistant tailored natively for deep Earth Observation (EO). The system interprets free-form natural language strings, intelligently maps intent across asynchronous satellite streams, performs precise pixel-level bounding-box and polygon segmentation, and outputs production-grade spatial payloads directly compatible with industry-standard GIS systems.

2. Strategic System Architecture

The system is engineered around a distributed, highly decoupled multi-layer paradigm to ensure processing efficiency, multi-sensor scalability, and sub-second textual interaction latencies:

- **Multi-Modal Ingestion Proxy & Parse Infrastructure:** Handles high-bitrate multi-band geospatial file extensions (.tif, .nc, .hdr). It extracts native spatial geotransforms, metadata parameters, and geographic coordinate frames while applying automated min-max tensor normalization arrays.
- **Dynamic Intent-Driven Agentic Router:** Implements optimized text analytics to scan input prompts for functional keywords. It programmatically bifurcates computational tasks into isolated execution channels—Optical Stream (high-res object analytics), SAR Stream (cloud-penetrating microwave backscatter arrays), or Temporal Stream (historical bi-temporal change matrices).
- **Dual-Stream Cross-Attention Token Alignment Engine:** Fuses disparate sensory information matrices (e.g., visual RGB frames and polarized radar signatures) by aligning tokens across a joint visual embedding canvas before routing values to the primary multi-modal language transformer.
- **Cascaded Deep-Feature Visual Grounding Interface:** Intercepts textual bounding-box indicators printed by the conversational Large Vision Model (LVM) and channels them as coordinate prompts straight into a **Segment Anything Model 2 (SAM 2)** instance to generate high-fidelity pixel segmentation masks.
- **Self-Correction Verifier & Spatial Projection Array:** Re-projects coordinate regions into accurate geographical representations via structural GIS tools (GDAL/Rasterio) while utilizing an automated topological verification loop to filter out system hallucinations.

Table 2.1: Structural Comparison of Legacy Platforms vs. SatQuery AI

Functional Layer	Legacy & SOTA Architectures (WIIS / WOIS / Static VLMs)	Next-Gen SatQuery AI Pipeline
Query Routing	Manual selection of bands or rigid, token-dependent single-prompt text decoding.	Dynamic NLP-driven Agentic Router allocating tasks to dedicated specialist paths.
Visual Grounding	Coarse bounding box texts or basic classification classes without polygons.	Cascaded Deep-Feature Alignment Head mapping LVM coordinates into SAM 2.
Data Support	Strictly single-sensor constraints (mostly low-resolution optical standard RGB).	Native cross-modal integration of high-res Optical, SAR, and temporal image pairs.
Output Payload	Plain standard chat updates or standalone web visualizations.	Production-ready, georeferenced vector arrays (GeoJSON / Shapefiles).
Trust Validation	Zero data checking; high vulnerability to generative text hallucinations.	Automated Self-Correction Verifier Loop matching masks to baseline parameters.

3. Production-Grade Technical Codebase

The executable script below sets up the foundational core engine for SatQuery AI. It features multi-modal cross-attention layers for sensor data alignment alongside text-driven intent routing algorithms:

```
# -*- coding: utf-8 -*-
import torch
import torch.nn as nn

class GeospatialCrossAttention(nn.Module):
    \"\"\"
    Production-grade Cross-Attention layer designed to align distinct sensor modalities.
    Fuses Optical texture tokens with SAR structural backscatter profiles.
    \"\"\"
    def __init__(self, feature_dim: int = 512, num_heads: int = 8):
        super(GeospatialCrossAttention, self).__init__()
        self.multihead_attention = nn.MultiheadAttention(embed_dim=feature_dim,
                                                         num_heads=num_heads,
                                                         batch_first=True)
        self.layer_norm_optical = nn.LayerNorm(feature_dim)
        self.layer_norm_sar = nn.LayerNorm(feature_dim)
        self.feed_forward = nn.Sequential(
            nn.Linear(feature_dim, feature_dim * 4),
            nn.ReLU(),
            nn.Linear(feature_dim * 4, feature_dim)
        )
        self.final_norm = nn.LayerNorm(feature_dim)

    def forward(self, optical_features: torch.Tensor, sar_features: torch.Tensor) -> torch.Ten
sor:
        # Normalize incoming inputs to stabilize gradients during cross-training
        norm_optical = self.layer_norm_optical(optical_features)
        norm_sar = self.layer_norm_sar(sar_features)
```

```
        # Calculate cross-attention weights across features
        attn_output, _ = self.multihead_attention(query=norm_optical, key=norm_sar, value=norm
_sar)
        interim_features = optical_features + attn_output

        # Apply token feed-forward network
        output_features = interim_features + self.feed_forward(self.final_norm(interim_feature
s))
        return output_features

class SatQueryIntentRouter:
    \"\"\"
    Dynamic context broker that parses natural language queries
    and routes processing tasks to the appropriate sensor pipelines.
    \"\"\"
    def __init__(self):
        self.routes = {
            "SAR_STREAM": ["flood", "water", "radar", "sar", "oil spill", "night"],
            "TEMPORAL_STREAM": ["change", "before", "after", "evolution", "growth", "history"]
,
            "OPTICAL_STREAM": ["count", "urban", "vegetation", "crop", "object", "aircraft", "
ship"]
        }

    def route_query(self, user_prompt: str) -> str:
        normalized_prompt = user_prompt.lower()
        if any(keyword in normalized_prompt for keyword in self.routes["TEMPORAL_STREAM"]):
            return "TEMPORAL_STREAM"
        if any(keyword in normalized_prompt for keyword in self.routes["SAR_STREAM"]):
            return "SAR_STREAM"
        return "OPTICAL_STREAM"
```

4. Strategic Dataset Strategy & Curation

To ensure high diagnostic accuracy across various geographical domains, the model training framework utilizes a multi-sensor dataset strategy:

- **Visual Question Answering & Grounding:** Utilizes **RSVLM-QA** (~162,000 distinct visual question-answering pairs across WHU and LoveDA datasets) and **VRSBench** (29,000 images mapped to 123,000 VQA items and 52,000 bounding boxes) to train the core conversational assistant.
- **High-Res Structural Detection:** Leverages **DIOR-RSVG** and **DOTA v2.0** to fine-tune the system's object localization capabilities for compact structures like aircraft, vessels, and storage tanks.
- **Dynamic Environmental Assessment:** Integrates **FloodNet** (high-res drone/satellite imagery of post-disaster infrastructure) alongside **LandCover.ai** to optimize land-cover classification and flood-extent mapping.

5. Deployment, Vector Ingestion & UI Integration

The system is packaged as an enterprise-grade containerized microservice using FastAPI backends and a React/Leaflet interactive frontend dashboard. When a user queries the interface, the backend computes the textual reasoning layout, passes coordinates to SAM 2, converts the output array to real-world coordinates via Rasterio, and returns a georeferenced GeoJSON package. This ensures the output can be added immediately as a layer onto live maps, providing an intuitive tool for rapid geospatial analysis.

Footnote: This is for informational purposes only. For medical advice or diagnosis, consult a professional. AI responses may include mistakes.